STRIPE PROJECT

1. OLTP Data System

Annexes

1.C. OLTP Supporting Notes

> *Nicolas Pichon - AIA RNCP 38777 / BC02 / OLTP Data System - Annexe C / v30 - 2026/10/13.*

1.C.1. Principes de conception

1.C.1.1. Fils conducteurs

Trois critères de décision qui reviennent systématiquement :

- ne jamais ralentir le chemin critique d'exécution lors d'une procédure de paiement (transaction),
  - découplage, écriture asynchrone, CDC vs. triggers ;
- préserver le cloisonnement des clients par marchand à tous les niveaux ;
- choisir la structure de données proportionnée à la richesse métier réellement portée par chaque information :
  - table quand il y a des métadonnées ou une logique d'état,
  - colonne simple sinon.

1.C.1.2. Principes structurants

Normalisation en 3NF

Le schéma élimine la redondance et garantit l'intégrité.
Chaque fait vit à un seul endroit, ce qui évite les incohérences lors des écritures massives.

Clés primaires en UUID

Génération distribuée sans point de contention central

- indispensable pour le futur *sharding* d'une table qui encaisse des millions d'écritures par jour.

`Transaction` comme entité auto-suffisante

Montant, devise et date permettent toute conversion *a posteriori* côté analytique.
Ce choix évite d'introduire une table de taux de change dans l'OLTP et réoriente la conversion vers l'OLAP,
avec un taux de référence figé plutôt que le taux du jour, pour rendre les montants convertis comparables entre eux.

1.C.1.3. Découplage autour de la transaction

`Refund`, `Chargeback`, `FraudScore` séparés de `Transaction`

Deux justifications combinées :
éviter des colonnes `NULL` massives sur une table à fort volume (la plupart des transactions n'ont ni remboursement ni litige),
et ne pas verrouiller la table transactionnelle avec des écritures tardives ou asynchrones
(un score de fraude s'écrit en temps réel par un service séparé, une rétrofacturation évolue sur plusieurs semaines).

`Refund` vs `Chargeback` : deux entités distinctes

Elles reflètent deux réalités opposées - le remboursement est une décision du marchand,
la rétrofacturation une contestation imposée par la chaîne bancaire.
Attributs, cycles de vie et acteurs diffèrent entièrement, d'où l'absence de mutualisation.

`FraudRiskLevel` : table de référence ajoutée en v25

`risk_level` (low | medium | high) était initialement laissé en simple
`CHECK`, au même titre que `device_type`. Une revue a montré que ce

n'était pas cohérent : dans un système de fraude réel, le niveau de risque déclenche une action différenciée (auto-approbation, revue manuelle, blocage). `FraudRiskLevel` porte donc `requires_manual_review` et `blocks_transaction`, permettant de dériver l'action à prendre plutôt que de la coder en dur côté application à chaque occurrence de `'high'`.

`FraudReview` et `FraudDecision` séparés de `Transaction` Même logique de découplage que pour `FraudScore` : une revue humaine est un événement asynchrone, déclenché parfois plusieurs jours après la transaction (échantillonnage, contestation), et une décision peut être révisée sans jamais toucher à la ligne `Transaction`. Séparer ces deux entités évite qu'une confirmation de revue — écriture différée, potentiellement lente si elle implique un processus humain — ne verrouille une table à fort volume et à forte contrainte de latence.

1.C.1.4. Cloisonnement Merchant / Customer

`Merchant` comme entité distincte de `Customer`

Le marchand porte un rôle réglementé - celui qui encaisse assume KYC/KYB, responsabilité en cas de litige, reversement des fonds - que le client n'a pas.

`Customer` modélise la relation client-marchand et non la personne physique

Une même personne achetant chez trois marchands produit trois lignes distinctes. Ce n'est pas un défaut mais une conséquence voulue de la confidentialité entre marchands concurrents, du RGPD (le consentement est donné à un marchand précis) et de la responsabilité de traitement. Le rapprochement transverse d'une identité, quand il est nécessaire, se fait dans une couche séparée (NoSQL), pas dans l'OLTP.

1.C.1.5. Gestion des énumérés

Tables de référence pour les énumérés porteurs de métadonnées

Tables concernées : `TransactionStatus`, `MerchantStatus`, `SubscriptionStatus`, `ChargebackStatus`, `ChargebackReasonCode`, `PaymentMethodType`, `CardBrand`.
Critère retenu : dès qu'une valeur porte une logique métier exploitable (`is_terminal`, `allows_refund`, `can_process_payments`, `triggers_dunning`) ou que la liste est vivante (codes de rétro-facturation normalisés par les réseaux, nouvelles marques de carte), une table s'impose.
Ajouter une valeur devient un `INSERT`, pas une migration de schéma.

Clé primaire naturelle et lisible (`'refunded'` non entière).

La colonne référençante reste interprétable sans jointure ; la jointure ne sert qu'à obtenir les métadonnées enrichies.

Contrainte CHECK conservée pour les listes courtes et figées

Une table dédiée serait disproportionnée pour ces liste sans métadonnée à porter.
Listes concernées :

- `Transaction.device_type` (mobile | desktop | tablet) :
  purement descriptif ; aucune note, aucune règle du MCD ni du modèle physique ne décrit une action différente selon la valeur.
- `Subscription.billing_cycle` (weekly | monthly | yearly) :
  même raison ; la seule métadonnée référencable, nombre de jours par cycle, est calculée côté application au moment du prélèvement.
- `ConsentRecord.purpose` (marketing | analytics | data_sharing | profiling)
  et `.source` (checkout | email | account_settings) : purement déclaratives ;
  qualifient l'enregistrement sans déclencher de comportement différent ;
  c'est `is_granted` qui porte la logique d'état).

- `ConsentRecord.legal_basis` (consent | contract | legitimate_interest) :
  cas limite ; le modèle ne porte actuellement aucune règle qui permettrait de distinguer le traitement selon cette valeur ; à surveiller.
- `DataSubjectRequest.request_type` (access | rectification | erasure | portability | objection) :
  chaque type a un traitement métier différent, mais cette différence vit dans le code applicatif, pas dans une donnée que le schéma aurait besoin de porter.
- `SecurityIncident.severity` (low | medium | high | critical)
  et `.incident_type` (unauthorized_access | data_breach | anomalous_activity | failed_auth_burst) :
  passage en référence plausible (escalade automatique, notification différenciée) mais non spécifié dans le cahier des charges.

1.C.1.6. Abonnements et Produits

Modélisation de l'abonnement comme entité

`Subscription` porte un cycle de vie propre (actif, en pause, en impayé (`past_due` , avec relance automatique), résilié) et relie client, produit et moyen de paiement autorisé pour le prélèvement récurrent.

Nullabilité de `Transaction.product_id` et `Transaction.subscription_id`

`Transaction.product_id` et `Transaction.subscription_id` doivent être nullables pour deux raisons :

- Permettre les transactions suivantes :
  - Achat ponctuel d'un produit du catalogue --> `product_id` renseigné, `subscription_id` nul.
  - Prélèvement récurrent généré par un abonnement --> `subscription_id` renseigné, `product_id` renseigné (car l'abonnement porte lui-même un produit).
  - Paiement libre, hors catalogue --> `product_id` et `subscription_id` sont nuls (cf. MCD/règle #6).
- Permettre que les indicateurs de suivi de performance produit (revenu total, nombre de ventes, popularité par produit ; cf. Business Case/ Data Source DS2 : "Product Performance Metrics") soient calculés par des agrégations simples du type " `SELECT product_id, SUM(amount)FROM Transaction GROUP BY product_id` ".
  Pour que cette requête soit à la fois directe (pas de jointure conditionnelle compliquée) et correcte (le total par produit reflète vraiment ce produit, ni plus ni moins),
  il faut que `product_id` soit renseigné directement sur `Transaction` pour toutes les ventes qui concernent un produit et nul pour celles ce qui n'en concernent aucun.

`Transaction.is_merchant_initiated`

Un prélèvement d'abonnement n'a ni clic, ni appareil, ni géolocalisation exploitables :
ce champ évite qu'un modèle de fraude calibré sur le comportement client ne signale systématiquement les paiements récurrents comme suspects.

1.C.1.7. Sécurité et conformité

Enregistrements des journaux en écriture seule

Les tables d'enregistrements d'évènement destinés aux audits de sécurité ou de conformité sont en écriture seule (suppression des droits sur `UPDATE` ni `DELETE`)
Tables concernées : `DataAccessLog` , `ChangeAuditLog` , `FraudScoringLog` , `FraudScore` , `ConsentRecord` .

Écriture asynchrone des journaux

Pour que l'enregistrement des journaux ne ralentissent pas les paiements, en doublant le coût de chaque écriture sur `Transaction` ,
les tables `DataAccessLog` et `ChangeAuditLog` doivent être écrites par des mécanismes asynchrones,
qui n'utilisent pas les déclencheurs TRIGGER (les *triggers* se déclenchent dans la même transaction que l'écriture d'origine).
`ChangeAuditLog` sera alimentée par un mécanique CDC (Change Data Capture), qui utilise un outil externe (ex: Debezium) pour lire le WAL (write-ahead log) du SGBD (PostgreSQL) *a posteriori*.
Un autre mécanisme asynchrone devra être mis en place pour tracer les *consultations* dans `DataAccessLog`
(par exemple, en publiant les évènements d'accès sur un bus d'évènements externe).

Double rattachement de `ConsentRecord` au client et au marchand

`ConsentRecord` est également rattaché au marchand car un client *consent* à un marchand précis et pas à l'intermédiaire *Stripe*.

Délais matérialisés comme données mesurables

Propriétés concernées : `DataSubjectRequest.due_at` / `fulfilled_at` , `SecurityIncident.detected_at` / `notified_at` .
Un rapport de conformité doit prouver le respect des délais légaux et pas seulement l'existence d'une procédure.

1.C.2. Justification du modèle de tarification

Origine du besoin

La paire de tables (`PricingPlan` , `PricingPlanFee`) capture la commission *Stripe* à la source pour permettre a posteriori le calcul des montants des commissions dans la devise de référence dans le modèle analytique (cf. Business Requirement BR2 : "revenue analysis", et Deliverable D8 : requêtes de revenu).

Pourquoi a-t-on besoin de deux tables?

`commission_rate` (un taux) et `fixed_fee` (un montant monétaire, dans une devise donnée) n'ont pas
la même relation à la devise : le taux, à l'inverse du montant de frais fixe, est indépendant de la devise.

Modéliser `fixed_fee` comme attribut scalaire de `PricingPlan` suppose que chaque plan à sa propre devise,
hypothèse qui devient fausse dès qu'un marchand traite plusieurs devises.
`PricingPlanFee` porte donc `fixed_fee` comme propriété d'une relation *n,n* entre `PricingPlan` et `Currency` :
chaque plan peut être libellé dans plusieurs devises, chacune avec son propre frais fixe, sans conversion de change nécessaire pour l'appliquer.

Pourquoi `PricingPlan` est-il versionné dans le temps?

`effective_from` / `effective_to` permettent à un marchand de renégocier sa commission
(ou de changer de catégorie de risque) sans réécrire l'historique :
les transactions passées restent rattachées au plan qui était réellement en vigueur à leur date.
Cette contrainte n'est pas exprimable dans le modèle physique
et doit être transcrite en contrainte applicative (ou en trigger) comme l'absence de chevauchement des plages `[effective_from, effective_to)` pour un même `merchant_id` .

Pourquoi `Transaction` porte-t-il à la fois `pricing_plan_id` et `fee_amount` , alors que `fee_amount` est dérivable de `pricing_plan_id` ?

Il s'agit de permettre à cette propriété d'être recalculable tout en traçant chaque valeur calculée pour figer l'historique des valeurs.
En particulier, lorsq qu'un plan tarifaire est corrigé rétroactivement, un écart entre la valeur stockée et la valeur recalculée devient lui-même un signal exploitable (audit, détection d'anomalie de facturation).

Contraintes de cohérence non exprimables dans le modèle physique

Pour une transaction donnée :

- le plan référencé par `pricing_plan_id` doit bénéficier au marchand de cette même transaction ==> (`Transaction.merchant_id == PricingPlan[Transaction.pricing_plan_id].merchant_id`);
- la valeur de `fee_amount` doit rester calculable à partir des lignes de `PricingPlan` et de `PricingPlanFee` correspondant à la transaction :
  - fee_amount[transaction.currency] = princing_plan_fee[princing_plan[transaction]; transaction.currency].fixed_fee + princing_plan[transaction].commission_rate * transaction.amount

1.C.3. Justification du modèle de sécurité et conformité

Origine du besoin

Les exigences *BR5* (Business Requirement) et *TR5* (Technical Requirement) exigent explicitement les fonctionalités suivantes :

- journalisation d'audit,
- conformité RGPD/PCI-DSS,

- et reporting de conformité automatisé.

Ces fonctions sont réalisées par les tables suivantes :
`DataAccessLog`, `ChangeAuditLog`, `FraudScoringLog`, `FraudScore`, `FraudReview`, `FraudDecision`, `ConsentRecord`, `DataSubjectRequest` et `SecurityIncident`.

Tables de référence `AuditActorType` et `AuditAction`

Pourquoi `AuditActorType` et `AuditAction` sont-elles des tables de référence, et non de simples `CHECK` sur des listes de valeurs figées?
Ces tables portent une logique métier qu'une contrainte déclarative ne peut pas exprimer :

- `is_human` distingue un accès humain d'un accès automatisé (pertinent pour la détection d'anomalie),
- `is_sensitive` et `requires_justification` alimentent l'alerting temps réel et l'obligation de motif.

Séparation des journaux `DataAccessLog` et `ChangeAuditLog`

Pourquoi `DataAccessLog` (lecture) et `ChangeAuditLog` (écriture) sont-elles deux tables distinctes?
Ces tables portent de nombreux attributs différents et les fusionner produirait des colonnes NULL à grande échelle
(`Refund` / `Chargeback` / `FraudScore` sont séparés de la table `Transaction` pour la même raison).
La fusion de ces table reste légitime dans le modèle analytique (reporting consolidé).

Pourquoi les tables de journalisation sont-elles en écriture seule (append-only)

Un journal modifiable ne prouve rien. En conséquence, les droits `UPDATE` et `DELETE` sont supprimés pour tout le monde, y compris pour les administrateurs,
sur les tables de journaux : `FraudScoringLog`, `DataAccessLog`, `ChangeAuditLog`.
La table `ConsentRecord` suit le même principe mais pour une raison réglementaire différente : un retrait de consentement doit créer une nouvelle ligne,
et ne pas écraser la précédente, car on doit conserver un historique complet des décisions du client.

Pourquoi les tables d'évènements `DataSubjectRequest` et `SecurityIncident` ne sont-elles pas en écriture seule?

Ce sont des entités de *workflow* dont le statut évolue légitimement dans le temps (`received` --> `fulfilled`, `open` --> `closed`).
Les dates de traitement complètent l'enregistrement initial et évitent de créer une nouvelle ligne à chaque étape du workflow:

- `due_at` encode directement l'échéance légale RGPD (un mois),
- l'écart entre `detected_at` et `notified_at` est le point de contrôle direct du délai RGPD de notification (72 heures).

Nullabilité de `merchant_id` ?

Il n'y a pas de marchand à référencer dans `DataAccessLog`, `ChangeAuditLog`, et `SecurityIncident`, lorsqu'un accès ou un incident est transverse à la plateforme,
et indépendant d'un marchand précis. À l'inverse, `merchant_id` ne doit pas être nul dans `ConsentRecord` et `DataSubjectRequest` car un client consent toujours à un marchand précis.

Articulation avec le reste du schéma

Le fait de tracer les 4 derniers chiffres du PAN dans `PaymentMethod.card_last4` participe également aux exigences de conformité avec PCI-DSS.

Tables de référence `Regulation`, `DataSubjectRequestStatus`, `SecurityIncidentStatus`

Application du principe de la section 1.C.1.5 :

- `DataSubjectRequestStatus` et `SecurityIncidentStatus` modélisent chacune un statut de workflow avec état terminal (`fulfilled` / `rejected` ou `closed`),
  comme `is_terminal` dans `TransactionStatus`, `SubscriptionStatus` et `ChargebackStatus`.
- `Regulation.response_deadline_days` représente un délai légal qui prend une valeur différente en fonction de la réglementation GDPR/CCPA.
  Avec cette information de référence, `DataSubjectRequest.due_at` devient dérivable de `received_at` + le délai légal.

1.C.4. Stratégie de réplication, failover et reprise après sinistre

1.C.4.1. Contexte

> Couvre un sujet ignoré du modèle OLTP :
>
> - *T1* exige des *mécanismes de réplication et de restauration de données en temps réel (real-time data replication and failover)*,
> - *BR1* exige explicitement une procédure de *recouvrement en cas de désastre" (disaster recovery)*.
>
> Ces deux points ne sont traités que par une ligne de recommandation dans les notes du schéma, complétés par le diagramme du modèle conceptuel (D2) et par les notes justificatives des choix de modélisation les plus significatifs.

1.C.4.2. Principes directeurs

Le système *OLTP* supporte l'écriture de différentes familles de données :

- Les données d'enjeu critique, financier ou contractuel : `Transaction`, `Subscription`, `Refund`, `Chargeback`, `FraudScore`, `FraudDecision`, `FraudScoringLog` ;
- Les journaux de sécurité et conformité (`DataAccessLog`, `ChangeAuditLog`, `SecurityIncident`, `ConsentRecord`, `DataSubjectRequest`, `FraudScoringLog`, `FraudReview`) ;
- etc.

Chaque famille correspond à des exigences de continuité propres :

Famille de tables	RPO (Recovery Point Objective)	RTO (Recovery Time Objective)
Enjeu critique (`Transaction`, `Subscription`, etc.)	~0 (aucune transaction confirmée ne doit être perdue)	Quelques secondes à ~1 minute
Référentiels (`Country`, `Currency`, statuts, ...)	Quelques minutes	Quelques minutes (lecture seule tolérable en repli)
Sécurité & Conformité (`DataAccessLog`, `ChangeAuditLog`, ...)	Jusqu'à ~1 minute (écriture déjà asynchrone par conception)	Quelques minutes

La disparité de ces exigences de continuité justifie une architecture à réplication *différenciée* plutôt qu'une politique unique pour toute la base.

1.C.4.3. Réplication en temps réel

Réplication physique en flux

Réplication physique en flux (streaming replication *PostgreSQL*) vers au moins deux répliques :

- 1 réplique *synchrone* dans la même région (zone de disponibilité différente) :
  - le commit d'une transaction sur `Transaction` n'est confirmé au client qu'une fois répliqué sur ce noeud (`synchronous_commit = on`, `synchronous_standby_names` ciblant ce noeud) ;
    c'est le mécanisme qui garantit `RPO ~ 0` sur l'écriture des données d'enjeu critique ;
- 1 ou plusieurs répliques *asynchrones* supplémentaires, dans une région géographique distincte :
  - l'objectif est d'absorber la perte d'une région entière (reprise après sinistre) sans pénaliser la latence d'écriture des données d'enjeu critique.

CDC (Change Data Capture)

CDC via réplication logique (`pgoutput` / Debezium) en parallèle de la réplication physique : c'est le mécanisme modélisé dans `ChangeAuditLog` et documenté comme alimentant OLAP/ NoSQL de façon asynchrone, hors chemin transactionnel.
La réplication physique assure la continuité de service. Le *CDC* logique assure la synchronisation vers les systèmes analytiques (couvre BR4/TR4).

1.C.4.4. Failover

Détection et bascule automatiques

Détection et bascule automatiques via un gestionnaire de haute disponibilité (Patroni + etcd/Consul, ou équivalent géré par le fournisseur cloud) :

élection d'un nouveau primaire parmi les répliques synchrones en cas de perte du noeud primaire, sans intervention manuelle.

Fenêtre de bascule cible

Fenêtre de bascule cible < 30 secondes (cohérente avec le RTO des données d'enjeu critique du tableau §1.C.4.2).

Proxy de connexion

Proxy de connexion (*PgBouncer*/*HAProxy* ou équivalent managé) devant le cluster, pour que les applications ne référencent jamais directement un noeud physique : la bascule reste transparente côté application.

Répliques asynchrones inter-régions

Les répliques asynchrones inter-régions ne participent pas à l'élection automatique (latence réseau trop variable pour un quorum fiable).
Elles ne servent qu'en cas de reprise après sinistre, déclarée manuellement (voir §1.4).

1.C.4.5. Reprise après sinistre (disaster recovery)

Sauvegardes physiques continues

Sauvegardes physiques continues (WAL archiving en continu + snapshot de base hebdomadaire, via pgBackRest ou équivalent managé), permettant une **restauration à un point dans le temps (PITR)** n'importe où dans la fenêtre de rétention (recommandé : 35 jours, aligné sur les exigences de délai RGPD/PCI-DSS déjà portées par `DataSubjectRequest.due_at`).

Bascule inter-région

Bascule inter-région déclarée manuellement vers la réplique asynchrone distante en cas de perte totale de la région primaire. Processus documenté et testé (exercice de bascule programmé, a minima trimestriel), plutôt qu'automatisé, pour éviter un failover inter-région intempestif sur un simple pic de latence réseau.

Intégrité vérifiable après bascule

`fee_amount` (`Transaction`) doit rester re-calculable à partir de `PricingPlan` / `PricingPlanFee`. Un écart après restauration signale une réplication incomplète plutôt qu'un plan tarifaire corrigé rétroactivement (cf. note déjà présente sur `Transaction`).

Tables d'audit jamais perdues silencieusement

Toute purge ou incohérence détectée sur `DataAccessLog` / `ChangeAuditLog` après un incident de bascule doit elle-même être tracée dans `SecurityIncident`, la table conçue pour capturer les incidents est aussi celle qui documente les incidents touchant l'infrastructure elle-même.

1.C.4.6. Couverture de la stratégie

- Couvre explicitement :
  - T1 (*"mechanisms for real-time data replication and failover"*),
  - BR1 (*"disaster recovery"*, *"real-time data synchronization"*),
  - TR3 (haute disponibilité). Reste hors périmètre de ce document, à traiter séparément : le choix précis du fournisseur/outillage managé (RDS Multi-AZ, Cloud SQL, Aurora...), qui est une décision d'infrastructure et non de modélisation de données.

1.C.5. Couverture des exigences

#	Exigence	Couverture	Source
BR1	haut volume, ACID, sync. temps réel, disaster recovery	🟢	Indexation `Transaction` ([D2-B]), découplage des entités dépendantes ([D2-B]), Disaster Recovery/Replication/Failover (§1.C.4)
BR2	analyse de revenu	🟢	`PricingPlan` / `PricingPlanFee` / `Transaction.fee_amount` (§1.C.2)
BR4	flux cohérent OLTP ↔ OLAP ↔ NoSQL	🟢 (part OLTP)	*CDC* via `ChangeAuditLog` ; écriture asynchrone hors du chemin d'éxecution critique
BR5/TR5	chiffrement, accès, audit, RGPD/PCI-DSS	🟢	audit & RGPD/PCI-DSS --> Tables conformité (§1.C.3)

#	Exigence	Couverture	Source
TR1	schéma normalisé, intégrité, performance	🟢	3NF, tables de référence porteuses de logique métier, index ciblés
TR3	scalabilité horizontale, indexation, partitionnement, sharding	🟢	Index existants ; stratégie de partitionnement/sharding formalisée au §1.C.4
TR4	cohérence et synchronisation (CDC)	🟢	`ChangeAuditLog` , réplication logique (cf. §1.C.4)

Couverture du modèle

Le choix précis de l'outillage *managé* (*RDS Multi-AZ*, *Cloud SQL*, *Aurora*...) est hors périmètre.
C'est une décision d'infrastructure/déploiement.

Références

Ressources

- [R0] Stripe Business Case
-

Annexes

- [D2-B] OLTP Physical Model